SP-103 - Visualizing Black-Box Models with SHAP and LIME

Final Report Draft

CS 4850 - Section 02 – Spring 2026

April 20, 2026

Team Members:

Name	Role
Shamir Howlader	Team Lead
Dang Tran	Dev/Doc
Phillip Pham	Dev/Doc
Sai Samudrala	Dev/Doc
Sharon Perry	Project Owner or Advisor

Status:
Statistics:

1. Project Overview 2
1.1 Abstract 2
1.2 Project Scope 2
1.3 Platform and Tools 2
2. Software Requirements 3
2.1 Functional Requirements 3
2.1.1 Data Input 3
2.1.2 Prediction 3
2.1.3 Explanation 3
2.1.4 Visualization 3
2.2 Non-Functional Requirements 3
2.3 Constraints 4
3. Software Design 5
3.1 Architectural Overview 5
3.2 Design Decisions 5
3.3 System Components 5
3.3.1 Data Management Subsystem 5
3.3.2 Model Training Subsystem 5
3.3.3 Explainability Subsystem 6
3.3.4 User Interface Subsystem 6
4. Implementation 7
4.1 System Integration & Workflow 7
4.2 Preprocessing 7
4.3 Model Training 8
4.4 Prediction & Explainability 9
4.4.1 SHAP 9
4.4.2 LIME 9
4.5 File Structure & Configuration 9
5. Testing 11
5.1 Test Scope 11
5.2 Test Objectives 11
5.3 Test Strategy 12
5.4 Defect Management 12
5.5 Environment Roles 12
6. Known Limitations & Future Work 14
6.1 Known Limitations 14
6.2 Future Work 14
7. References & Glossary 15
7.1 References 15

7.2 Glossary 15

1. Project Overview

1.1 Abstract

This project develops an interactive Explainable AI system that enables users to understand and interpret predictions made by complex black-box machine learning models. Using the UCI Adult Census Income dataset, an XGBoost classifier was trained to predict whether an individual earns more than $50,000 per year. SHAP and LIME are used to generate both global and local explanations, showing how individual features influence specific predictions.

The primary goal is to reduce the transparency in black-box AI by providing visualizations and quantitative explanations. The system is implemented in Python using Jupyter Notebooks and is planned for deployment as a Streamlit-based web application.

1.2 Project Scope

The system accomplishes the following:

- Trains an XGBoost binary classifier for income classification (<=50K or >50K per year)
- Accepts user-uploaded CSV data for batch predictions and analysis
- Generates global feature importance explanations (whole model behavior)
- Generates local explanations for individual predictions
- Visualizes results using SHAP summary plots, waterfall plots, force plots, bar plots, scatter plots, and LIME plots
- Will be deployed as a Streamlit web application

1.3 Platform and Tools

Category	Details
Hardware	Standard laptop/desktop, multicore CPU, 8GB+ RAM (no GPU required)
Language	Python 3.11+
ML Libraries	scikit-learn, XGBoost 1.7.6
Explainability	SHAP 0.51.0, LIME 0.2.0.1
Data Handling	pandas, numpy
Visualization	matplotlib
UI	Streamlit
Dev Environment	Jupyter Notebook, VS Code, Git/GitHub
Dataset	UCI Adult Census Income (adult.csv, ~32,000 entries)

2. Software Requirements

2.1 Functional Requirements

2.1.1 Data Input

The system shall:

- Allow manual data entry via an input form corresponding to model features (age, education, occupation, etc.)
- Allow CSV dataset upload for batch predictions and analysis
- Validate uploaded file format and dataset structure
- Provide sample data for demonstration purposes

2.1.2 Prediction

- Pass user input through the trained XGBoost model for income classification
- Display income classification results (<=50K or >50K)
- Display prediction confidence/probability scores

2.1.3 Explanation

- Generate SHAP global feature importance plots (summary, bar)
- Generate SHAP local plots (waterfall, force, scatter)
- Generate LIME local explanations for individual predictions
- Allow users to view both SHAP and LIME explanations side by side

2.1.4 Visualization

- Display predictions in graphical format
- Allow viewing of both global and local explanations
- Present a user-friendly dashboard with clear labeling

2.2 Non-Functional Requirements

Category	Requirement
Capacity	Support multiple users; generate explanations within reasonable time (prediction <10s, single-row SHAP/LIME <5s)
Usability	Intuitive UI for users with basic technical knowledge; explanations in plain language
Reliability	Handle invalid inputs with informative error messages (wrong format, wrong structure, empty file, etc.)
Portability	Runs on Windows 10/11, CPU-only (no GPU required)
Reproducibility	random_state=42 used throughout; identical results on re-run

2.3 Constraints

- Input datasets must be in CSV format and conform to the model's trained feature schema
- Limited to classification tasks supported by XGBoost and scikit-learn
- Larger datasets will increase SHAP/LIME computation time
- Must be operational within the course semester timeline
- No external API keys or databases required

3. Software Design

3.1 Architectural Overview

The system follows a layered architecture:

1. User Interface Layer – handles user interaction and visualization (Streamlit)
2. Application Logic Layer – data validation, model execution, and explanation generation
3. Machine Learning Layer – performs predictions using the pretrained XGBoost pipeline
4. Explainability Layer – generates SHAP (global + local) and LIME (local) explanations

The data flow through the system is: User Input (manual entry or CSV upload) → Preprocessing / Validation → XGBoost Model → Prediction Output → SHAP & LIME Explainers → Visualizations displayed to the user.

3.2 Design Decisions

Decision	Reason
sklearn Pipeline	Unifies preprocessing and XGBoost model so identical transformations apply to training and testing
XGBoost	High accuracy on structured/tabular data; native compatibility with SHAP TreeExplainer
SHAP explainer	Produces Shapley values; supports both global and local explanations
LIME explainer	Provides comparative local explanations
Streamlit for UI	Python-native, easy integration with ML visualizations; no HTML/CSS/JS knowledge needed
joblib serialization	Pipeline saved to xgb_pipeline.joblib; explainability notebook runs independently without retraining

3.3 System Components

3.3.1 Data Management Subsystem

- load_dataset(file) – loads and validates uploaded CSV files
- preprocess_data(data) – cleans and encodes dataset features
- split_data(data) – splits data into training and testing sets
- Accepts only .csv format; assumes tabular data with labeled target variables

3.3.2 Model Training Subsystem

- train_model(data, model_type) – trains the XGBoost binary classifier
- predict(model, input_data) – returns income class predictions and probabilities
- Model saved via joblib.dump() to models/xgb_pipeline.joblib

3.3.3 Explainability Subsystem

- generate_shap_explanations(model, data) – produces global and local SHAP explanations
- generate_lime_explanations(model, instance) – generates local LIME explanations for individual predictions

3.3.4 User Interface Subsystem

- Streamlit-based web interface for dataset upload and manual input
- Visualization dashboard for predictions and SHAP/LIME explanations

4. Implementation

4.1 System Integration & Workflow

The system integrates a full machine learning pipeline: data preprocessing, XGBoost model training, and explainability via SHAP and LIME. The pipeline is designed to be reproducible. The operational workflow is:

1. Load saved XGBoost pipeline and test data
2. Transform raw test features using the stored ColumnTransformer preprocessor
3. Generate binary income predictions (<=50K or >50K) using the XGBoost classifier
4. Apply SHAP TreeExplainer to compute global and local feature importance scores
5. Apply LIME LimeTabularExplainer to generate local explanations for individual predictions
6. Display visualizations: SHAP summary, waterfall, force, bar, scatter plots; LIME feature-weight list

4.2 Preprocessing

The dataset used is the UCI Adult Income dataset (~32,000 entries), containing demographic and employment features. Key preprocessing steps:

- fnlwgt column is dropped (irrelevant weighting factor)
- Missing values labeled '?' are replaced with NaN
- education column dropped in favor of education.num

- ColumnTransformer applies OneHotEncoder to all categorical columns; numeric columns pass unchanged
- Preprocessor fits on training data only and is stored inside the sklearn Pipeline

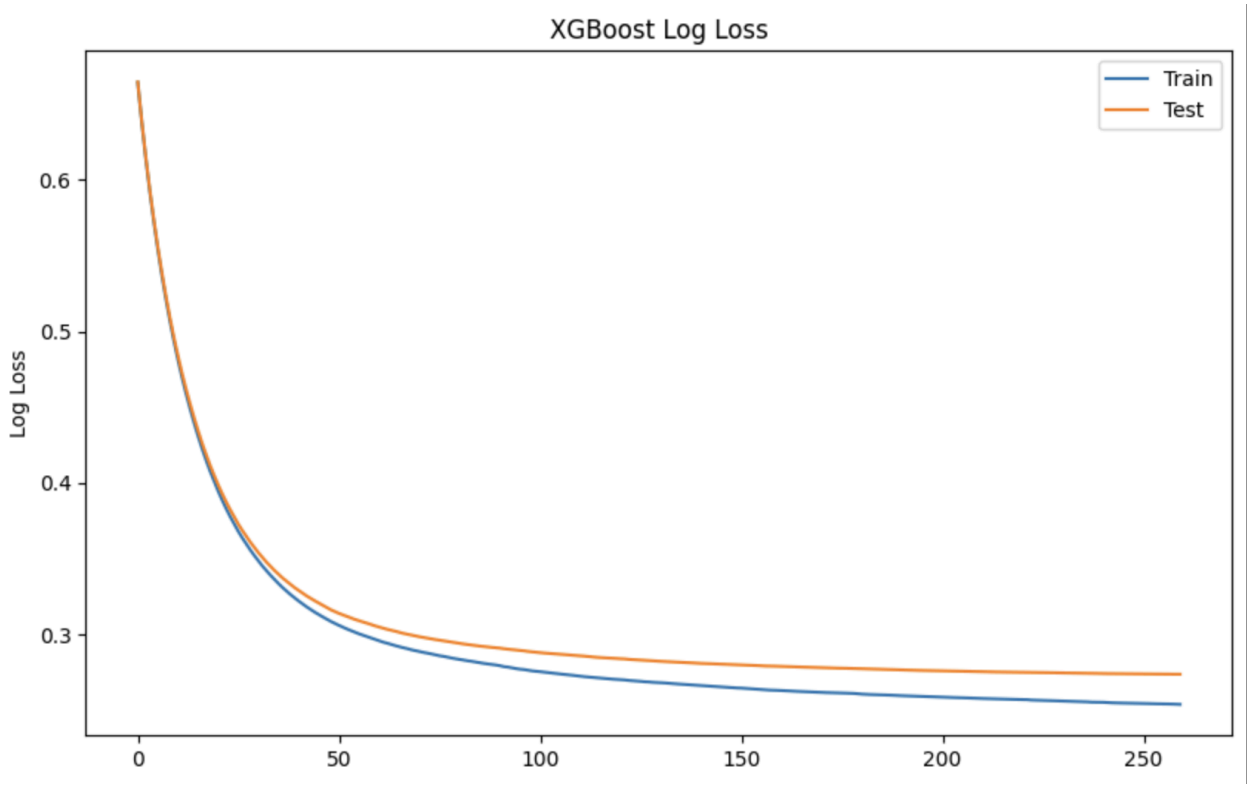
Feature	Type	Description
age	Numerical	Age of individual
education.num	Numerical	Years of education (numeric encoding)
capital.gain	Numerical	Capital gains from investments
capital.loss	Numerical	Capital losses
hours.per.week	Numerical	Weekly working hours
workclass	Categorical (OHE)	Type of employment
marital.status	Categorical (OHE)	Marital status
occupation	Categorical (OHE)	Job category
relationship	Categorical (OHE)	Relationship status
race	Categorical (OHE)	Race
sex	Categorical (OHE)	Sex
native.country	Categorical (OHE)	Country of origin

4.3 Model Training

The XGBoost classifier is wrapped inside a sklearn Pipeline to unify preprocessing and prediction into a single object. Hyperparameters used:

- n_estimators: 260
- max_depth: 6
- learning_rate: 0.05
- objective: binary:logistic

An 80/20 train-test split is applied with random_state=42. The model is evaluated with an eval_set containing both training and test transformed data to monitor logloss over epochs. A logloss plot is generated to confirm convergence with minimal overfitting.



Plot 1: Logloss Plot over 260 epochs (n_estimators = 260)

Metric	Class: <=50K	Class: >50K	Overall
Precision	~0.89	~0.79	—
Recall	~0.95	~0.62	—
F1-Score	~0.92	~0.70	—
Accuracy	—	—	~87%

4.4 Prediction & Explainability

4.4.1 SHAP

SHAP is initialized using shap.TreeExplainer, which is optimal for tree-based models. It uses the processed feature matrix and outputs probability-based SHAP values per feature. Visualizations include:

- Summary Plot: global feature importance across all test observations, colored by feature value magnitude
- Waterfall Plot: how each feature pushes a single prediction above or below the base value
- Force Plot: feature contributions as a horizontal diagram for individual or all predictions
- Bar Plot: mean absolute SHAP value per feature as a ranked bar chart
- Scatter Plot: SHAP values for a specific feature vs. its raw value, showing non-linear relationships

SHAP identifies capital.gain, capital.loss, education.num, and age as the top global predictors for income classification, consistent with feature expectations.

4.4.2 LIME

LIME is applied using LimeTabularExplainer, which fits a locally weighted linear model around a specific instance. The explainer is initialized with processed training data, one-hot encoded feature names, and class names. For a given person, lime_explainer.explain_instance() returns the top contributing features and their weights, printed as a ranked feature-weight list.

4.5 File Structure & Configuration

- data/adult.csv – raw input dataset (UCI Adult Income)
- data/X_test.csv – test features (saved after training split)
- data/y_test.csv – test labels (income column, 0 or 1)
- models/xgb_pipeline.joblib – saved sklearn Pipeline with fitted preprocessor and XGBoost model
- Training.ipynb – loads raw data, runs preprocessing/training, evaluates, writes pipeline and test data to disk
- Explainability.ipynb – loads trained pipeline and runs SHAP/LIME independently; does not retrain
- Explainability_methods.py – turns notebook logic into reusable functions of SHAP/LIME
- App.py – user interface implementation using Streamlit
- requirements.txt – all dependency versions pinned for reproducibility

No API keys or external database connections are required. All file paths are resolved dynamically using Python's pathlib.Path relative to the notebook root.

5. Testing

5.1 Test Scope

The test plan covers the trained XGBoost classifier, SHAP global and local interpretations, and LIME local explanations. The Streamlit UI is currently out of scope pending completion.

In Scope:

- XGBoost model training validation (accuracy, precision, recall, F1)
- SHAP global feature importance consistency across test samples
- SHAP local explanation correctness for individual predictions
- LIME local explanation stability on test data
- Cross-validation of SHAP and LIME feature rankings for directional agreement
- Streamlit UI testing (planned for next milestone)
- Load/stress testing of the web interface

Out of Scope (current release):

- Database or API integration testing (not applicable)

5.2 Test Objectives

Ref	Function	Test Objective	Evaluation Criteria
PT-01	Data Preprocessing	Verify pipeline drops fnlwgt, replaces '?' with NaN, encodes categorical features	No fnlwgt; no unknown-category errors
PT-02	Model Training	Confirm XGBoost achieves >=87% accuracy with defined hyperparameters	Test accuracy ~87%; precision/recall within thresholds
PT-03	Model Serialization	Verify model saves/reloads correctly via joblib without prediction drift	Reloaded pipeline produces identical predictions to in-memory model
PT-04	SHAP Explainer	Validate SHAP values computed without error; numerically consistent with model output	All plot types render; predicted values make sense
PT-05	LIME Explainer	Validate LIME produces locally faithful, directionally consistent explanations	Explanation object returned; LIME weights align directionally with SHAP
PT-06	Streamlit UI Upload	Verify UI accepts CSV, runs pipeline, displays predictions with scores	Results table displayed; no startup or pipeline errors
PT-07	Streamlit Error Handling	Confirm UI surfaces informative error messages for malformed uploads	Specific error messages for missing columns, bad types, empty files
PT-08	Performance	Verify response times within acceptable limits	Prediction table <10s; single-row SHAP/LIME explanation <5s

5.3 Test Strategy

Test Type	Objectives
Functional / Progression	Verify each feature behaves per specification; accuracy thresholds met; SHAP/LIME return correct outputs
Negative Testing	Confirm invalid inputs produce informative errors rather than silent failures or unhandled exceptions
Regression Testing	Re-execute full test suite after any change to hyperparameters, preprocessing, or library versions
Performance Testing	Verify prediction and explanation workflows complete within accepted time limits

Test Type	Objectives
Cross-Method Agreement	Run SHAP and LIME on same samples; confirm top feature rankings agree directionally

5.4 Other Testing

5.4.1 Security

5.4.2 Stress & Volume Testing (S&V)

5.4.3 Connectivity Testing (CT)

5.4.4 Disaster Recovery/Backup

5.4.5 Unit Testing

5.4.6 Integration Testing

5.5 Defect Management

Defects are tracked within the project documentation. The process is:

- Any team member who identifies a failure raises a defect entry, including test case reference, severity, reproduction steps, and resolution notes
- Critical defects are escalated immediately for prioritization
- Defects are assigned to the developer responsible for the failing component; fixes are reviewed by at least one other team member
- Defects may not be closed without a confirmed PASS on re-test

5.6 Environment Roles

Role	Assigned To	Responsibilities
Release Manager	Sai Samudrala	Overall coordination and support of the test environment
Test Manager	Phillip Pham / Dang Tran	Advise release manager of environment requirements
Project Manager	Shamir Howlader	Escalation point for environment and defect issues

6. Known Limitations & Future Work

6.1 Known Limitations

- The dataset is heavily imbalanced (~75% <=50K class), reflected in the lower recall for the >50K class (~0.65) and a low SHAP base value (E[f(X)] = 0.047). No upsampling or class weighting was applied during training.

- Hyperparameters (n_estimators, max_depth, learning_rate) were manually selected and may not represent the optimal configuration. Automated tuning (grid search) could improve performance.
- Due to one-hot encoding, many categorical features are split into sub-categories (e.g., oneshot_relationship_wife), making SHAP and LIME output less human-readable feature names.

6.2 Future Work

- Apply upsampling or class weighting to improve recall for the minority >50K class
- Implement automated hyperparameter tuning for better model performance
- Extend support to additional ML model types (e.g., Random Forest, gradient boosting variants)

7. References & Glossary

7.1 References

#	Document / Resource	Notes
1	Project Plan Document	Sprint goals and team charter
2	Software Design Document (SDD)	Architecture and subsystem design
3	Software Requirements Spec (SRS)	Functional and non-functional requirements
4	Project Development Document	Implementation details and build state
5	scikit-learn documentation	https://scikit-learn.org/stable/
6	SHAP documentation	https://shap.readthedocs.io/en/latest/
7	LIME documentation	https://lime-ml.readthedocs.io/en/latest/
8	XGBoost documentation	https://xgboost.readthedocs.io/en/stable/
9	Streamlit documentation	https://streamlit.io/
10	UCI Adult Income Dataset	https://archive.ics.uci.edu/ml/datasets/adult

7.2 Glossary

Term / Acronym	Definition
Black-box model	Machine learning system where internal decision-making logic is not visible or interpretable
CSV	Comma-Separated Values file format
joblib	Python serialization library used to save and reload the trained XGBoost pipeline

Term / Acronym	Definition
LIME	Local Interpretable Model-Agnostic Explanations – local explanation technique using locally weighted linear models
ML	Machine Learning
OHE	One-Hot Encoding – encoding categorical variables as binary columns
scikit-learn	Open-source Python machine learning library
SHAP	SHapley Additive exPlanations – game-theoretic approach to feature attribution
Streamlit	Python framework for building interactive web applications
UI	User Interface
XAI	Explainable Artificial Intelligence – methods that make AI decision-making transparent
XGBoost	Optimized distributed gradient boosting library designed for efficiency and performance